

WEEK – 7: METHOD OVERRIDING

Bank (Dynamic Polymorphism)

Experiment No.	07	Date	08/10/2026
Title	Method Overriding – Bank (Dynamic Polymorphism)		

1. Objective / Aim

To write a Java program to demonstrate method overriding and dynamic polymorphism using a bank example. Different bank classes override the `getRateOfInterest()` method to provide different interest rates. The appropriate method is called at runtime using a parent class reference.

2. System Requirements

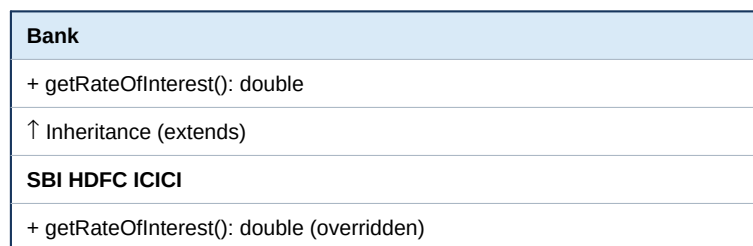
Hardware: Standard PC/laptop, Intel Core i3/i5 or equivalent processor, minimum 4 GB RAM, and 500 MB free disk space.

Software: Windows/Linux/macOS, JDK 17 or JDK 21, Eclipse/IntelliJ IDEA/NetBeans/VS Code, and Command Prompt or Terminal.

3. Algorithm / Logic Description

1. Start the program.
2. Create a parent class named Bank with a method `getRateOfInterest()`.
3. Create subclasses SBI, HDFC, and ICICI that extend Bank.
4. Override the method in each subclass with a different interest rate.
5. Create a parent class reference for each subclass object.
6. Call the overridden method and display each bank's interest rate.
7. Stop the program.

4. UML Class Diagram



5. Complete Java Source Code

Save the file as **BankDemo.java**.

```
class Bank {
    double getRateOfInterest() {
        return 0.0;
    }
}

class SBI extends Bank {
    @Override
    double getRateOfInterest() {
        return 7.0;
    }
}

class HDFC extends Bank {
    @Override
    double getRateOfInterest() {
        return 7.5;
    }
}
```

```

}

class ICICI extends Bank {
    @Override
    double getRateOfInterest() {
        return 8.0;
    }
}

public class BankDemo {
    public static void main(String[] args) {
        // Parent class reference to child class objects
        Bank b;

        b = new SBI();
        System.out.println("SBI Rate of Interest: "
            + b.getRateOfInterest() + "%");

        b = new HDFC();
        System.out.println("HDFC Rate of Interest: "
            + b.getRateOfInterest() + "%");

        b = new ICICI();
        System.out.println("ICICI Rate of Interest: "
            + b.getRateOfInterest() + "%");
    }
}

```

6. Compilation and Execution Commands

Compile: javac BankDemo.java

Run: java BankDemo

7. Output Screen

```

SBI Rate of Interest: 7.0%
HDFC Rate of Interest: 7.5%
ICICI Rate of Interest: 8.0%

```

Note: These interest rates are illustrative values for demonstrating method overriding, not current bank deposit rates.

8. Real-Time Applications

1. Banking systems: Different banks may provide different interest rates.
2. Payment applications: Different payment methods can implement their own processing behavior.
3. E-commerce: Different payment options can use different implementations.
4. Employee management: Different employee types can calculate salaries differently.
5. Game development: Different characters can override common actions.

9. Result

Thus, the Java program demonstrating method overriding and dynamic polymorphism using a bank example was implemented and executed successfully. The overridden method is selected at runtime according to the actual object.